

**MÉTODOS NUMÉRICOS II - COMPUTACIÓN CIENTÍFICA**
**LIC. EN INFORMÁTICA -ING. EN INFORMÁTICA**
Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán
**AÑO 2026**

# Proyecto 6: Factorización LU Paralela (A = LU)

*Eliminación gaussiana con pivoteo parcial en MPI: análisis de speedup, escalabilidad y precisión del error residual*

**Integrantes**
Naessens, Maia Guadalupe — Ingeniería en Informática
Martínez, Ezequiel — Ingeniería en Informática

**Año 2026**

# Índice

**Índice** ..... 2
**1. Resumen** ..... 3
**2. Introducción** ..... 3
**3. Marco teórico** ..... 4
**3.1. Factorización LU y eliminación gaussiana** ..... 4
**3.2. Pivoteo parcial y estabilidad numérica** ..... 4
**3.3. Residuo, error y condicionamiento** ..... 5
**3.4. Costo computacional y dónde está el paralelismo** ..... 5
**3.5. MPI y modelo de costos** ..... 5
**4. Datos y Métodos** ..... 6
**4.1. Algoritmo implementado: cíclico por filas** ..... 6
**4.2. Decisiones de diseño y reparto de carga** ..... 6
**4.3. Resolución del sistema y error residual** ..... 6
**4.4. Entorno de hardware y software** ..... 7
**4.5. Metodología de medición** ..... 7
**5. Resultados y discusión** ..... 8
**5.1. Validación numérica** ..... 8
**5.2. Escalabilidad fuerte: speedup y eficiencia** ..... 8
**5.3. Escalabilidad del tiempo con N** ..... 10
**5.4. Balance de carga: reparto cíclico vs. por bloques** ..... 11
**5.5. Distinta capacidad de procesamiento: PC vs. cluster** ..... 12
**5.6. Precisión del error residual** ..... 12
**5.7. Variabilidad entre corridas** ..... 13
**6. Limitaciones y trabajo futuro** ..... 14
**7. Conclusiones** ..... 15
**8. Referencias** ..... 16

## 1. Resumen

En este trabajo se implementa y analiza la factorización LU con pivoteo parcial, el núcleo de la resolución de sistemas de ecuaciones lineales A·x = b en su versión paralela con memoria distribuida mediante MPI. El objetivo, siguiendo la consigna, es investigar la implementación paralela de la eliminación gaussiana y estudiar cómo escalan el tiempo de cómputo y la comunicación al aumentar la dimensión N, evaluando el speedup y la precisión numérica del error residual ‖A·x − b‖.

Se desarrolló una versión serial de referencia y una paralela con distribución cíclica por filas: las filas se reparten una sola vez y permanecen locales durante toda la factorización, de modo que en cada paso solo se comunican la fila pivote (MPI_Bcast) y el pivote global (MPI_Allreduce con MPI_MAXLOC). Así la comunicación es $O(N^2)$ frente a un cómputo $O(N^3)$, lo que da speedup real y creciente con el tamaño del problema.

Las mediciones se realizaron en un nodo de cómputo con 128 núcleos físicos y 503 GiB de RAM, del que utilizamos hasta 48 núcleos y 192 GB, y para contrastar hardware, en una notebook de 6 núcleos. Con N = 12000 y 48 procesos se alcanzó un speedup de 42,3 (eficiencia del 88 %), y la eficiencia mejora al crecer N, como predice el modelo T_com/T_comp ∼ p/N. El error residual relativo se mantuvo entre $10^{-12}$ y $10^{-10}$ e idéntico para cualquier cantidad de procesos, lo que confirma que la paralelización no introduce error numérico. Se compara además el reparto cíclico con el reparto por bloques, mostrando que el balance de carga es el factor que determina la eficiencia.

## 2. Introducción

La resolución de sistemas de ecuaciones lineales A·x = b es una de las operaciones más frecuentes de la computación científica: aparece en el análisis de circuitos, en el método de elementos finitos, en el ajuste por mínimos cuadrados y en el paso lineal interno de prácticamente todo método iterativo no lineal (por ejemplo, el paso de Newton). Cuando A es una matriz densa, el método estándar es la factorización LU, que descompone A en el producto de una matriz triangular inferior L (con diagonal unitaria) y una triangular superior U.

La factorización LU no es un método nuevo: es la eliminación gaussiana de siempre, guardando los multiplicadores que normalmente se descartan. Su ventaja es la reutilización: una vez obtenidas L y U, resolver el sistema para cualquier lado derecho cuesta solo $O(N^2)$, frente al $O(N^3)$ de la factorización. En problemas reales N no es 3 sino del orden de miles o millones, y el costo $O(N^3)$ convierte a la factorización en un candidato natural para la computación de alto rendimiento (HPC).

La factorización LU es, además, el núcleo del benchmark HPL (High Performance Linpack) que ordena la lista TOP500 de las supercomputadoras más potentes del mundo, lo que da una idea de su relevancia como problema paralelo. Las bibliotecas de producción (LAPACK en memoria compartida, ScaLAPACK en

memoria distribuida) implementan variantes por bloques y esquemas de comunicación reducida. Este trabajo implementa una versión didáctica pero completa, suficiente para estudiar los fenómenos centrales de la escalabilidad.

**Objetivos.** (a) Implementar la factorización LU con pivoteo parcial en forma serial y paralela con MPI. (b) Analizar cómo escalan el tiempo de cómputo y la comunicación al incrementar N. (c) Medir el speedup y la eficiencia, y relacionarlos con el modelo de costos. (d) Verificar la precisión numérica mediante el error residual ‖A·x − b‖. (e) Comparar estrategias de reparto de carga y distintas capacidades de hardware.

# 3. Marco teórico

## 3.1. Factorización LU y eliminación gaussiana

Los sistemas triangulares son fáciles de resolver: un sistema triangular inferior se resuelve hacia adelante y uno superior hacia atrás, una incógnita por vez, con costo $O(N^2)$. La idea de la factorización LU es transformar el sistema difícil en dos triangulares fáciles. Escribiendo $A = L \cdot U$, el sistema $A \cdot x = b$ se parte en dos:

$$L \cdot y = b \ \text{(sustitución hacia adelante)} \quad y \quad U \cdot x = y \ \text{(sustitución hacia atrás).}$$

En el paso k de la eliminación (k = 0, …, N−2), para cada fila i > k se calcula el multiplicador $l_{ik} = a_{ik} / a_{kk}$ y se actualiza la submatriz restante: $a_{ij} \leftarrow a_{ij} - l_{ik} \cdot a_{kj}$ para j > k. Al terminar, U ocupa la diagonal y la parte superior de A, mientras que los multiplicadores $l_{ik}$ forman L en la parte inferior. Esto permite almacenar L y U *in-place* en el mismo arreglo, sin memoria adicional. El costo total, sumando $2 \cdot (N-k-1)^2$ operaciones sobre todos los pasos, es de $(2/3) \cdot N^3$ operaciones de punto flotante.

## 3.2. Pivoteo parcial y estabilidad numérica

El algoritmo anterior falla si el pivote $a_{kk}$ es cero, y produce resultados malos (sin avisar) si es muy pequeño, porque dividir por un número chico genera multiplicadores enormes que, al sumarse en punto flotante, borran la información de las otras filas. La solución es el pivoteo parcial: antes de dividir, se busca el elemento de mayor valor absoluto en la columna k (de la fila k hacia abajo) y se lo lleva a la diagonal intercambiando filas. Intercambiar ecuaciones no cambia la solución, y garantiza que todos los multiplicadores cumplan $|l_{ik}| \leq 1$.

**Ejemplo.** Para el sistema $0{,}0001 \cdot x + y = 1$; $x + y = 2$, cuya solución es $x \approx 1$, $y \approx 1$, resolver con tres dígitos significativos sin pivoteo da x = 0 (100 % de error), porque el multiplicador 1/0,0001 = 10000 hace desaparecer los demás términos. Con pivoteo (se intercambian las filas porque $|1| > |0{,}0001|$) se obtiene x = 1, y = 1. La matriz está bien condicionada: lo que falla sin pivoteo es el algoritmo, no el problema. Con pivoteo parcial el teorema es completo: toda matriz invertible admite una factorización $P \cdot A = L \cdot U$, donde P es una matriz de permutación.

La matriz de permutación P nunca se almacena como una matriz de N×N (sería $O(N^2)$ con casi todos ceros): se guarda como un vector de enteros *pivotes*, donde pivotes[k] es la fila con la que se intercambió la fila k. Registrar un intercambio es O(1).

### 3.3. Residuo, error y condicionamiento

El error residual se define como $r = \|A\cdot\hat{x} - b\|$, donde $\hat{x}$ es la solución calculada. Un residuo del orden del epsilon de máquina ($\varepsilon \approx 2{,}2\cdot10^{-16}$ en doble precisión) significa que $\hat{x}$ es la solución exacta de un sistema levemente perturbado: es la *estabilidad backward*, lo máximo que se le puede pedir a un algoritmo. Sin embargo, un residuo chico no garantiza que $\hat{x}$ esté cerca de la solución verdadera: el error relativo de la solución está acotado por el número de condición, $\|\hat{x} - x\|/\|x\| \lesssim \kappa(A)\cdot$(residuo relativo). El condicionamiento $\kappa(A)$ es una propiedad del problema, no del algoritmo. Este trabajo reporta el residuo, que es la métrica que pide la consigna y la que mide la calidad del algoritmo paralelo.

### 3.4. Costo computacional y dónde está el paralelismo

La factorización cuesta $(2/3)\cdot N^3$ y las dos sustituciones solo $2\cdot N^2$, por lo que para N grande el 99,9 % del trabajo está en la factorización: es allí donde conviene paralelizar. Dentro de un paso k, la actualización $a_{ij} \leftarrow a_{ij} - l_{ik}\cdot a_{kj}$ es independiente para cada (i, j): son $(N-k-1)^2$ operaciones que no dependen unas de otras, un paralelismo de datos puro. Lo que no se puede paralelizar es el lazo sobre k, el paso k+1 necesita la matriz ya actualizada por el paso k, ni la elección del pivote dentro del paso. Por lo tanto, el paralelismo está dentro de cada paso, y los N pasos imponen N barreras de sincronización.

### 3.5. MPI y modelo de costos

MPI (Message Passing Interface) es el estándar para memoria distribuida: se ejecuta el mismo programa en varios procesos independientes (modelo SPMD), cada uno con su propia memoria, que se identifican por su *rank* y se comunican por mensajes. Este trabajo usa comunicación colectiva: MPI_Allreduce con la operación MPI_MAXLOC (que devuelve el valor máximo y el índice donde está, en una sola operación) para el pivoteo distribuido; MPI_Bcast para difundir la fila pivote; y MPI_Sendrecv_replace para intercambiar filas sin riesgo de deadlock.

**Modelo de costos.** Repartiendo las N filas entre p procesos, cada proceso realiza aproximadamente 1/p del trabajo total, así que el cómputo por proceso es $T_{comp} \approx (2/3)\cdot N^3/(p\cdot\gamma)$, con $\gamma$ la velocidad de la máquina en flops por segundo. La comunicación está dominada por la difusión de la fila pivote en cada uno de los N pasos: como nuestra implementación difunde la fila completa (N valores), el volumen total de datos comunicados es del orden de $N\cdot N = N^2$, y con difusiones en árbol (log p etapas) el tiempo es $T_{com} \approx N^2\cdot\log(p)\cdot\beta$, donde $\beta$ es el costo por dato transmitido. El cociente de ambos da la relación central del análisis:

$$T_{com} / T_{comp} \sim p / N.$$

Es decir: con N creciente y p fijo la comunicación se diluye y la eficiencia mejora; con p creciente y N fijo la comunicación pesa más y el speedup se aplana. El parámetro que gobierna la escalabilidad no es N ni p por separado, sino N/p: cuántas filas le tocan a cada proceso. A su vez, la Ley de Amdahl establece que la fracción serial del programa, aquí el overhead de comunicación y sincronización, que crece con p, impone una cota superior al speedup.

# 4. Datos y Métodos

## 4.1. Algoritmo implementado: cíclico por filas

Las filas se reparten de forma **cíclica** (round-robin): el proceso rank es dueño de las filas cuyo índice global g cumple g mod p = rank, y esas filas permanecen locales durante toda la factorización. En cada paso k:

- Pivoteo parcial distribuido: cada proceso busca el mayor $|a_{gk}|$ entre sus filas locales con $g \geq k$; con MPI_Allreduce(MPI_MAXLOC) se obtiene el pivote global y la fila que lo contiene.
- Intercambio de filas: las filas k y pivote las poseen a lo sumo dos procesos, que las intercambian con un único MPI_Sendrecv_replace (o un intercambio local si es el mismo proceso).
- Difusión de la fila pivote: su dueño la difunde a todos con MPI_Bcast. Es la única comunicación de datos del paso.
- Actualización local: cada proceso actualiza solo sus filas $g > k$, sin devolver nada.

De este modo, la comunicación total es $O(N^2)$ frente al cómputo $O(N^3)$, y el trabajo de actualización (el 99 % del tiempo) se reparte entre los procesos.

## 4.2. Decisiones de diseño y reparto de carga

Se eligió el reparto cíclico y no el *por bloques* (filas contiguas) por una razón de balance de carga: como la eliminación consume la matriz de arriba hacia abajo, con reparto por bloques los procesos dueños de las primeras filas quedan ociosos a mitad de camino. Con reparto cíclico, la diferencia de carga entre procesos nunca supera una fila. Se implementaron ambos repartos en el mismo programa para poder compararlos experimentalmente (sección 5.4). Otras decisiones: almacenamiento in-place de L y U (memoria $N^2$ en vez de $3 \cdot N^2$), vector de pivotes en vez de matriz de permutación ($O(N)$ en vez de $O(N^2)$), y que cada proceso reserve solo sus filas más un buffer para la fila pivote.

## 4.3. Resolución del sistema y error residual

Terminada la factorización, el proceso 0 reúne las filas (gather) para formar la matriz LU compacta, aplica la permutación al vector b y resuelve $A \cdot x = b$ por sustitución hacia adelante y hacia atrás. Luego calcula el residual $\|A \cdot x - b\|$ (norma euclídea, absoluta y relativa a $\|b\|$) usando una copia de la matriz y el vector

originales. Esta etapa de verificación es serial y queda fuera de la medición de tiempo, que abarca exclusivamente la factorización (la parte que se paraleliza).

## 4.4. Entorno de hardware y software

| Elemento | Cluster | Notebook (PC) |
|---|---|---|
| **Núcleos físicos** | 128 | 6 (12 hilos) |
| **Memoria RAM** | 503 GiB | ~16 GB |
| **Procesos usados** | 1 – 48 | 1 – 6 |
| **Sistema / MPI** | Linux, OpenMPI | Ubuntu (WSL), OpenMPI |
| **Compilación** | mpicc / gcc, sin optimización | mpicc / gcc, sin optimización |

*Tabla 1. Entornos de ejecución. Del nodo del cluster (128 núcleos, 503 GiB) utilizamos hasta 48 núcleos y 192 GB. No usa planificador de colas: se ejecuta con mpirun directamente.*

Se usó la implementación OpenMPI. El código se compiló sin opciones de optimización (-O0) para que las mediciones reflejen el algoritmo y no las transformaciones del compilador, y con las mismas banderas en la versión serial y en la paralela para que la comparación de speedup sea válida.

## 4.5. Metodología de medición

- La matriz A y el vector b se generan con valores aleatorios en [−1, 1] con una semilla fija, de modo que el problema es reproducible y el residual depende solo de N.
- El tiempo se mide con MPI_Wtime, precedido de un MPI_Barrier para que todos los procesos arranquen parejos. Se excluyen la generación de datos, el reparto, la resolución y el cálculo del residual.
- Cada configuración se ejecuta varias veces (5 repeticiones en el cluster, 20 en la PC) y se reporta la mediana, más robusta que el promedio ante corridas anómalas por la carga del sistema, siguiendo la metodología de la cátedra.

El tiempo de referencia $T_1$ para el speedup es el de la versión **serial pura** (el programa lu_serial). Se verificó que su valor es prácticamente idéntico al de la versión MPI con un solo proceso (N = 800: 0,2255 s el serial frente a 0,2257 s con p = 1), ya que con un único proceso no hay comunicación real; se toma el del serial por ser el tiempo del algoritmo secuencial, que es la referencia correcta y evita inflar el speedup.

- El límite de procesos se fija según los núcleos físicos (no los hilos lógicos), ya que OpenMPI asigna un slot por núcleo físico; correr más procesos que núcleos invalidaría el speedup.

# 5. Resultados y discusión

## 5.1. Validación numérica

Antes de medir rendimiento se verificó la correctitud. Para una misma matriz, el residual obtenido por la versión serial y por la versión paralela es idéntico para cualquier cantidad de procesos (p = 1, 2, 3, …). Esto ocurre porque el orden de las operaciones aritméticas sobre cada elemento es el mismo sin importar qué proceso lo ejecute: lo único que cambia es quién hace la cuenta. Que el residual no cambie con p confirma que la paralelización no introduce error numérico; si cambiara, sería señal de un error de programación.

## 5.2. Escalabilidad fuerte: speedup y eficiencia

Se midió el speedup $S_p = T_1/T_p$ y la eficiencia $E_p = S_p/p$ en el cluster, para N = 4000, 8000 y 12000, con p de 1 a 48. Los resultados muestran un speedup casi lineal que mejora al aumentar N.

| p | N=4000 S_p | N=4000 E_p | N=8000 S_p | N=8000 E_p | N=12000 S_p | N=12000 E_p |
|---|---|---|---|---|---|---|
| **1** | 1,00 | 100 % | 1,00 | 100 % | 1,00 | 100 % |
| **2** | 1,99 | 99,3 % | 1,99 | 99,6 % | 1,99 | 99,6 % |
| **4** | 3,98 | 99,6 % | 3,98 | 99,4 % | 3,98 | 99,5 % |
| **8** | 7,85 | 98,2 % | 7,88 | 98,4 % | 7,90 | 98,7 % |
| **16** | 14,74 | 92,1 % | 15,25 | 95,3 % | 15,43 | 96,5 % |
| **24** | 20,90 | 87,1 % | 22,42 | 93,4 % | 22,81 | 95,1 % |
| **32** | 26,29 | 82,1 % | 29,24 | 91,4 % | 29,39 | 91,8 % |
| **48** | 35,28 | 73,5 % | 40,72 | 84,8 % | 42,31 | 88,1 % |

*Tabla 2. Speedup y eficiencia en el cluster (mediana de 5 repeticiones).*

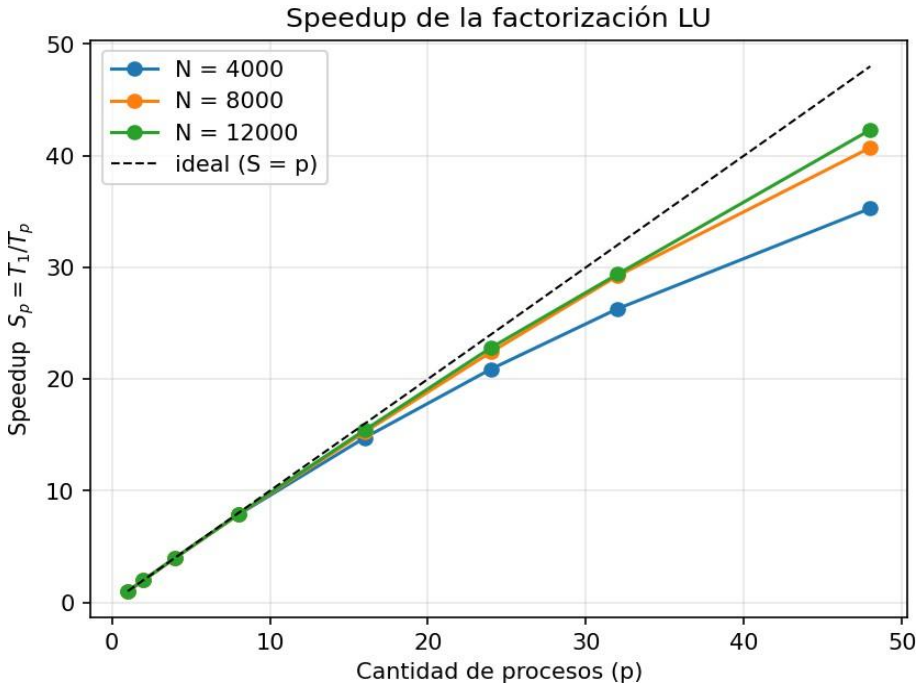



*Figura 1. Speedup vs. número de procesos.*

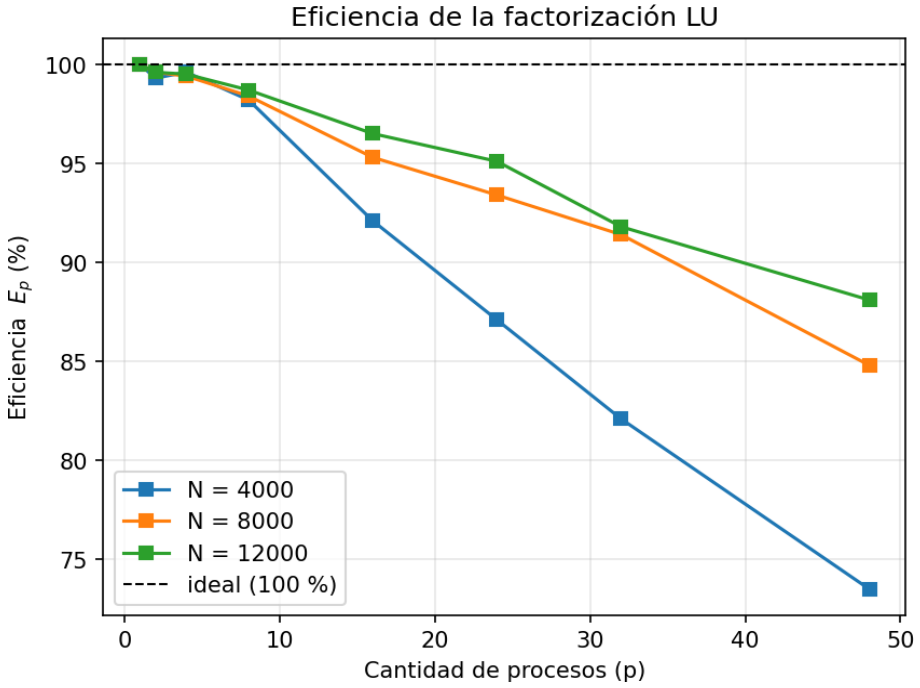



*Figura 2. Eficiencia vs. número de procesos.*

Hasta 8 procesos la eficiencia se mantiene en torno al 98 %, casi ideal: el cómputo domina ampliamente sobre la comunicación. Al aumentar p la eficiencia disminuye, por tres causas simultáneas: el overhead de comunicación (que crece con p), la parte serial del pivoteo (Ley de Amdahl) y el desbalance de carga. Lo más importante es que, a p fijo, la eficiencia mejora con N: con p = 48 pasa de 73,5 % (N = 4000) a 88,1 % (N = 12000). Esto es exactamente lo que predice el modelo $T_{com}/T_{comp} \sim p/N$: con más filas por proceso, la comunicación pesa relativamente menos. Es la razón por la que HPC tiene sentido para problemas grandes.

## 5.3. Escalabilidad del tiempo con N

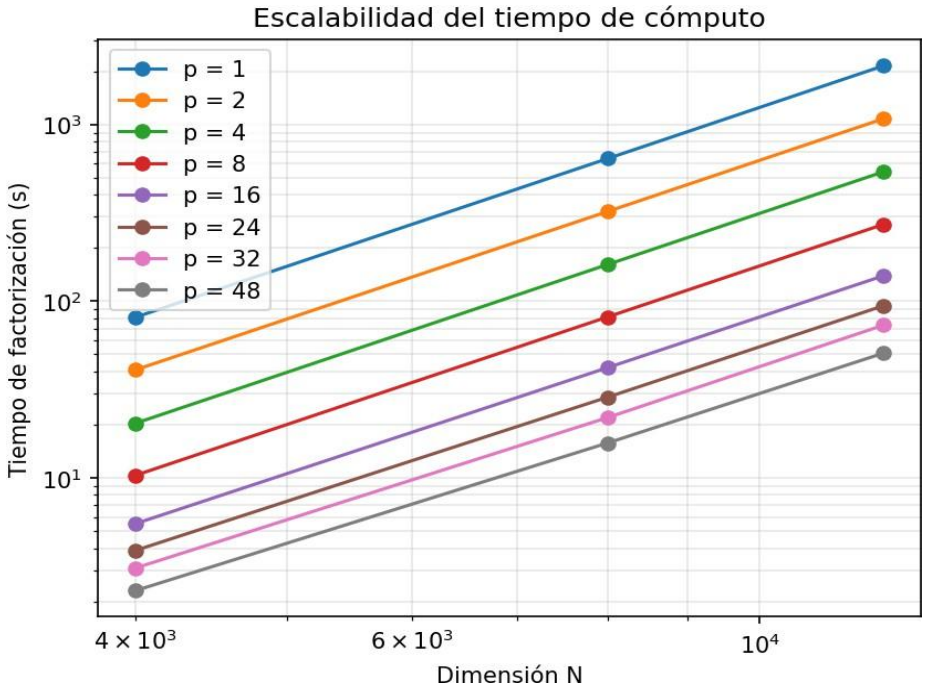



*Figura 3. Tiempo de factorización vs. N (log-log).*

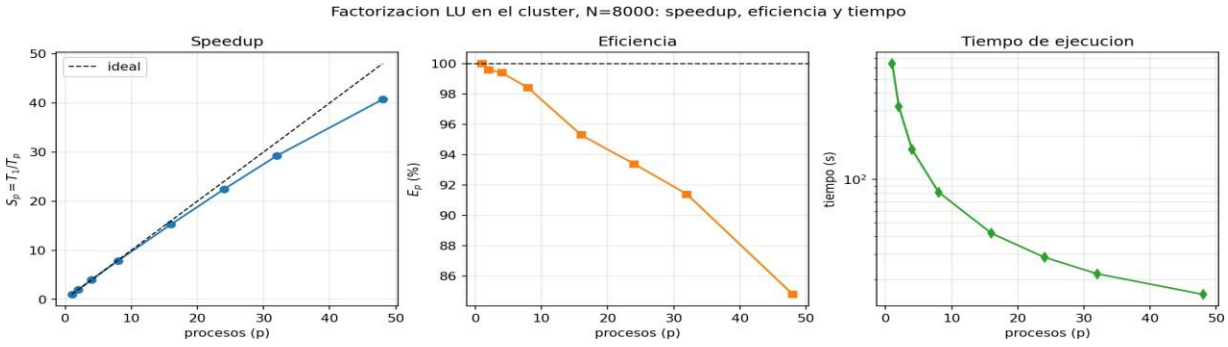



*Figura 4. Para N = 8000: speedup, eficiencia y tiempo de ejecución en función de p.*

En escala log-log el tiempo crece con pendiente cercana a 3, confirmando el costo $O(N^3)$: al duplicar N el tiempo se multiplica por unas 8 veces (N = 4000 con 1 proceso tarda 81 s; N = 8000, 641 s). La Figura 4 reúne, para un caso (N = 8000), las tres métricas: el speedup sube hacia la recta ideal, la eficiencia baja suavemente y el tiempo de ejecución cae de 641 s con 1 proceso a 15,7 s con 48. Speedup, eficiencia y tiempo son tres formas equivalentes de leer el mismo resultado.

## 5.4. Balance de carga: reparto cíclico vs. por bloques

Para mostrar el efecto del balance de carga se instrumentó el programa para medir por separado el tiempo de cómputo, de comunicación y ocioso (idle), y se compararon los dos repartos. El idle se aísla con una barrera antes de cada colectiva, por lo que estas corridas no se usan para el speedup sino solo para la composición del tiempo.

| Reparto | p | Cómputo (s) | Comunicación (s) | Idle (s) | Total (s) |
|---|---|---|---|---|---|
| **cíclico** | 2 | 17,18 | 0,064 | 0,101 | 17,37 |
| **bloques** | 2 | 17,17 | 0,061 | 6,499 | 23,75 |
| **cíclico** | 48 | 0,698 | 0,209 | 0,217 | 1,126 |
| **bloques** | 48 | 0,698 | 0,219 | 0,783 | 1,707 |

*Tabla 3. Composición del tiempo para N = 3000 (cluster).*

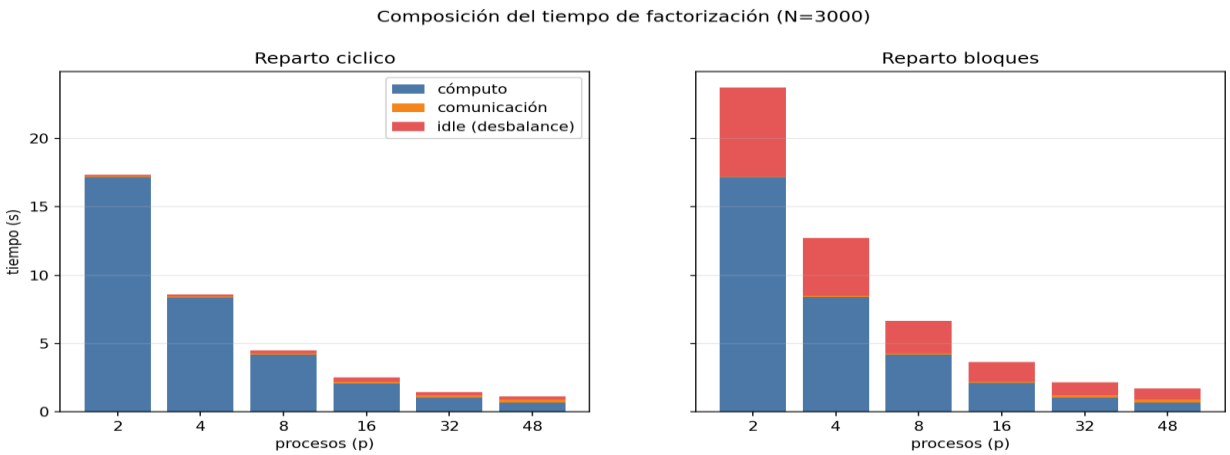



*Figura 5. Composición del tiempo, reparto cíclico vs. por bloques, N = 3000, hasta 48 procesos (cluster).*

El resultado es contundente: con el mismo cómputo y la misma comunicación, el reparto por bloques acumula un tiempo ocioso enorme (6,5 s con 2 procesos, frente a 0,10 s del cíclico) y por eso es mucho más lento (23,8 s vs. 17,4 s). La causa es clara: con 2 procesos el reparto por bloques le asigna a un proceso la mitad superior de la matriz, que se termina de eliminar a mitad de la factorización; ese proceso queda ocioso el resto del tiempo. El reparto cíclico, en cambio, mantiene a todos los procesos con filas activas hasta el final, y su idle es mínimo. La comunicación (naranja) es, en ambos casos, una fracción diminuta

del total: el factor que limita la escalabilidad no es la comunicación sino el balance de carga. La misma tendencia se verificó en la notebook con N = 1500 y hasta 6 procesos.

## 5.5. Distinta capacidad de procesamiento: PC vs. cluster

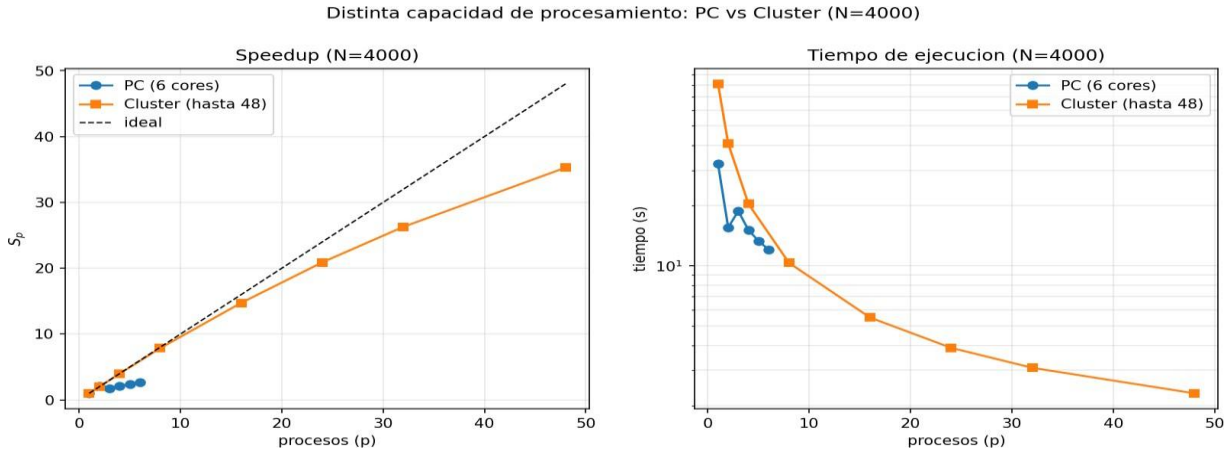



*Figura 6. PC (6 núcleos) vs. cluster (hasta 48), N = 4000: speedup (izq.) y tiempo de ejecución (der.).*

Comparar los dos equipos sobre el mismo problema (N = 4000) revela un matiz importante: la notebook tiene núcleos más rápidos (con 1 proceso tarda 32 s frente a los 81 s del cluster), pero está limitada a 6 núcleos. El cluster parte de un tiempo mayor por core pero escala hasta 48 procesos, de modo que a partir de p ≈ 8 lo supera y, con 48 procesos, resuelve el sistema en 2,3 s. Es la diferencia entre 'cores más rápidos' y 'muchos más cores': para problemas grandes, la capacidad de escalar gana.

## 5.6. Precisión del error residual

| N | ‖A·x – b‖ (absoluto) | Residual relativo |
|---|---|---|
| **4000** | $3{,}0 \cdot 10^{-10}$ | $8{,}2 \cdot 10^{-12}$ |
| **8000** | $1{,}2 \cdot 10^{-9}$ | $2{,}3 \cdot 10^{-11}$ |
| **12000** | $7{,}4 \cdot 10^{-9}$ | $1{,}2 \cdot 10^{-10}$ |

*Tabla 4. Error residual en función de N (idéntico para todo p).*

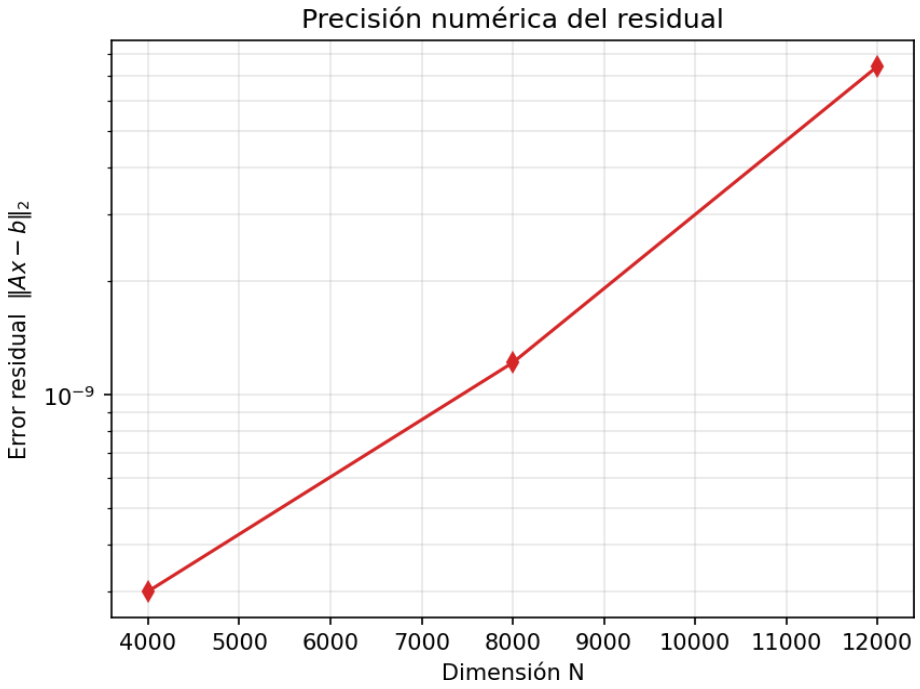



*Figura 7. Error residual absoluto ||A·x − b|| en función de N, con el eje vertical en escala logarítmica.*

La Figura 7 muestra cómo crece el residual con N. El eje vertical está en escala logarítmica porque los valores son muy pequeños y de órdenes distintos; en esa escala, una recta indicaría un crecimiento exponencial, pero acá la subida es suave. El residual relativo va de $8 \cdot 10^{-12}$ (N = 4000) a $1{,}2 \cdot 10^{-10}$ (N = 12000): sigue siendo diminuto y apenas aumenta con N.

Ese crecimiento moderado es esperable y no es un error: la cota de error de la eliminación gaussiana es proporcional al tamaño N y al número de condición de la matriz, que para matrices aleatorias crece con N; además, un cómputo de $O(N^3)$ operaciones acumula más error de redondeo. El pivoteo parcial mantiene el algoritmo estable. Como se explicó en la sección 3.3, un residuo pequeño indica estabilidad backward: la solución calculada es la solución exacta de un sistema muy cercano al original. El residual es idéntico para cualquier cantidad de procesos, lo que confirma que la paralelización no agrega error numérico.

## 5.7. Variabilidad entre corridas

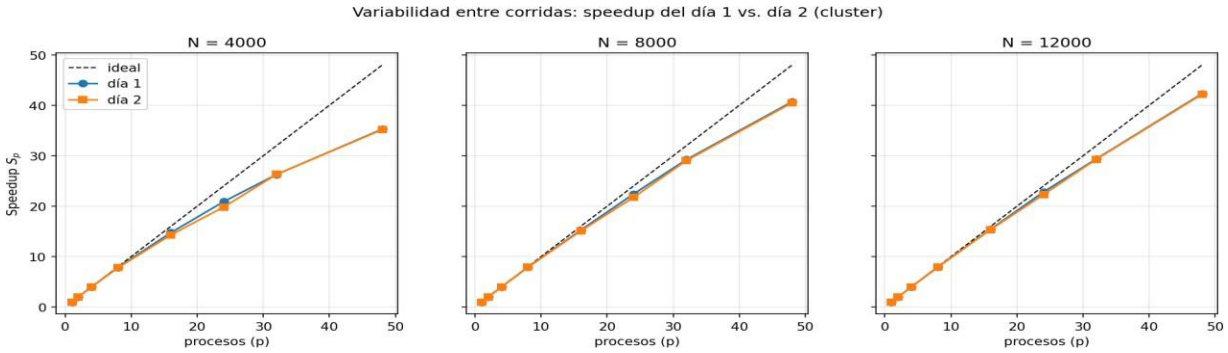



*Figura 8. Speedup del día 1 vs. día 2 para la misma configuración (cluster).*

Para evaluar la estabilidad de las mediciones, la barrida completa se repitió en el cluster en dos días distintos. Los residuales son idénticos entre ambas corridas (la aritmética no depende de la carga del sistema), pero el speedup muestra pequeñas diferencias: hasta cerca de un 5 % en los valores de p intermedios (por ejemplo, con N = 4000 y 24 procesos, 20,9 frente a 19,8). El motivo es que el nodo es un recurso compartido: la contención por el ancho de banda de memoria y la actividad de otros procesos varían de una corrida a otra, y con ellas el tiempo ocioso. La tendencia general es estable y las curvas casi se superponen, lo que confirma que una única ejecución no alcanza: es imprescindible repetir cada punto y reportar la mediana, como se hizo en todo el trabajo.

## 6. Limitaciones y trabajo futuro

La implementación es deliberadamente directa, y conocer sus límites es parte del análisis:

- Matriz en el proceso 0: la generación, el reparto (scatter) y la reunión (gather) para el residual hacen que el proceso 0 almacene la matriz completa ($N^2$). Funcionó holgadamente hasta N = 12000 (1,15 GB sobre 503 GiB), pero limita problemas muy grandes. La solución estándar es generar cada fila de forma independiente (generador sin estado) y calcular el residual de forma distribuida, de modo que ningún proceso tenga nunca la matriz completa.
- Comunicación: se difunde la fila pivote completa; difundir solo desde la columna k reduciría a la mitad el volumen. El tiempo informado es el del proceso 0; lo riguroso es el máximo entre procesos (MPI_Allreduce con MPI_MAX).
- Algoritmo escalar: nuestra implementación trabaja fila por fila (operaciones vector-matriz). Un algoritmo por bloques, que elimine de a varias columnas y actualice la submatriz restante con un producto matriz-matriz, reutilizando los datos en la memoria caché, sería un orden de magnitud más rápido con el mismo número de operaciones. Es la técnica que usan las bibliotecas optimizadas como LAPACK.

- Escalabilidad a cientos de procesos: una distribución bidimensional de la matriz (en una grilla de procesos, como hace ScaLAPACK) reduciría el costo de comunicación, que con nuestro reparto 1D por filas involucra a los p procesos en cada difusión.

## 7. Conclusiones

**A partir del trabajo realizado, se concluye lo siguiente:**

La factorización LU es un problema que justifica plenamente el uso de computación de alto rendimiento: su costo de cómputo crece como $O(N^3)$, de modo que al duplicar la dimensión N el tiempo se multiplica por ocho. Para matrices grandes, una sola máquina deja de ser suficiente, tanto en tiempo como en memoria, y paralelizar deja de ser un lujo para volverse una necesidad. El trabajo confirmó que la paralelización rinde cuando el volumen de cómputo supera ampliamente al costo de comunicación.

La implementación paralela desarrollada es correcta: el error residual $\|Ax - b\|$ resultó idéntico al de la versión serial para cualquier cantidad de procesos, lo que demuestra que repartir el trabajo no introduce error numérico. La clave para obtener aceleración fue una decisión de diseño: repartir las filas una sola vez y mantenerlas locales, comunicando en cada paso únicamente la fila pivote. Al quedar la comunicación en $O(N^2)$ frente a un cómputo $O(N^3)$, se alcanzó un speedup casi lineal, de hasta 42,3 con 48 procesos para N = 12000, con una eficiencia del 88 %.

El análisis de escalabilidad mostró que el rendimiento no lo determina N ni la cantidad de procesos p por separado, sino su relación N/p: la cantidad de filas que le toca a cada proceso. La eficiencia mejora al aumentar N y disminuye al aumentar p, exactamente como predice el modelo de costos $T_{com}/T_{comp} \sim p/N$.

La comparación entre el reparto cíclico y el reparto por bloques fue concluyente: con el mismo cómputo y la misma comunicación, el reparto por bloques acumula entre 3 y 64 veces más tiempo ocioso (dependiendo la cantidad de procesos). En este algoritmo, el factor que limita la eficiencia no es la comunicación sino el balance de carga, y elegir bien cómo se distribuyen los datos resultó más importante que cualquier optimización de los mensajes.

En cuanto a la precisión, el pivoteo parcial mantuvo el algoritmo numéricamente estable: el error residual se conservó en el orden del error de redondeo, creció muy lentamente con N por el condicionamiento de las matrices y fue independiente de la cantidad de procesos.

Por último, la comparación entre equipos mostró que el beneficio de paralelizar depende del hardware disponible: la notebook es más rápida por núcleo, pero el cluster, con muchos más núcleos, escala mejor

para problemas grandes. La variabilidad observada entre corridas confirmó, además, la importancia de repetir cada medición y reportar la mediana en lugar de una única ejecución.

**En términos prácticos:**

- Paralelizar conviene cuando el cómputo ($O(N^3)$) supera ampliamente a la comunicación ($O(N^2)$); cuanto mayor es N, mayor es el beneficio.
- El reparto cíclico por filas es el adecuado para la eliminación gaussiana, porque minimiza el tiempo ocioso por desbalance.
- La escalabilidad la gobierna N/p: hay que mantener suficientes filas por proceso para que el cómputo domine sobre la comunicación.
- Conviene usar a lo sumo un proceso por núcleo físico.
- Toda medición de rendimiento debe repetirse y resumirse con un estadístico robusto como la mediana.

**En síntesis,** el proyecto permitió implementar y verificar la factorización LU paralela, y comprender de primera mano que paralelizar no es automático ni siempre conveniente: su éxito depende del equilibrio entre cómputo, comunicación y balance de carga. Más allá del speedup obtenido, el principal aprendizaje fue que un buen rendimiento paralelo nace de buenas decisiones de diseño, cómo se distribuyen los datos y qué se comunica, y que esas decisiones solo pueden validarse midiendo.

## 8. Referencias

1. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities.
2. Burden, R. L., Faires, J. D. y Burden, A. M. (2016). Análisis numérico (10.ª ed.). Cengage Learning.
3. Cátedra de Métodos Numéricos II y Computación Científica. (2026). Computación de alto rendimiento para aplicaciones numéricas [Material de clase]. Facultad de Ciencias Exactas y Tecnología, Universidad Nacional de Tucumán.
4. Gropp, W., Lusk, E. y Skjellum, A. (2014). Using MPI: Portable parallel programming with the Message-Passing Interface (3.ª ed.). MIT Press.
5. Message Passing Interface Forum. (2021). MPI: A Message-Passing Interface Standard, Version 4.0.
6. Open MPI Project. (s. f.). Open MPI v5.0.x documentation.
7. Anthropic. (2026). Claude (Opus 4.8) [Asistente conversacional]. Consultado para apoyo en la redacción, aclaraciones conceptuales y síntesis teórica sobre factorización LU, programación paralela con MPI y análisis de escalabilidad.